

RELATÓRIO DE AUDITORIA DE SEGURANÇA

HizabellAi Music — Plataforma Transacional & Landing Page

Data da Auditoria:	10 de Setembro de 2026
Organização / Projeto:	tecflowsolucoess / hizabellai-music
Ambiente em Produção:	https://hizabellai-music.vercel.app
Escopo Auditado:	Código-fonte completo (index.html, api/*.js, vercel.json, .htaccess, histórico Git)
Classificação:	DOCUMENTO RESTRITO / CONFIDENCIAL

Nota Metodológica & Stack Tecnológica Detectada

Esta auditoria foi realizada com base na análise estática de código-fonte (SAST) e modelagem de ameaças, sem intervenção ou modificação de arquivos de código da aplicação. Antes da revisão das vulnerabilidades, a stack do projeto foi mapeada com exatidão:

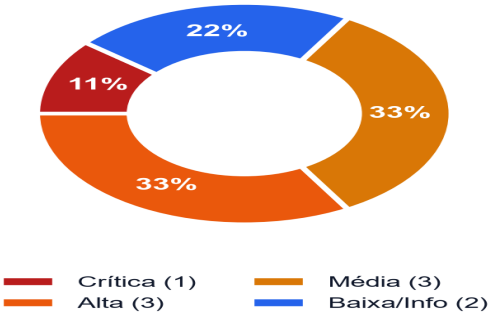
- Linguagem & Backend:** JavaScript (Node.js ESM) rodando em arquitetura Serverless Functions na Vercel (AWS Lambda sob o capô).
- Frontend:** Single Page Application (HTML5, TailwindCSS via CDN, Vanilla JavaScript puro no navegador).
- Mecanismo de Persistência / Banco:** Armazenamento baseado em arquivos JSON (/tmp/hizabellai_orders.json e data/orders.json), sem banco SQL/NoSQL ou RLS nativo.
- Autenticação & Autorização:** Checkout transacional sem contas de usuário; identificação por ID de pedido (orderId) e token estático para validação de webhook Asaas.
- Infraestrutura & Deploy:** vercel.json (Vercel Edge/Serverless) e regras de compatibilidade Apache em .htaccess.

Cada uma das 5 categorias mandatárias da auditoria foi estritamente adaptada e avaliada no contexto real desta stack tecnológica.

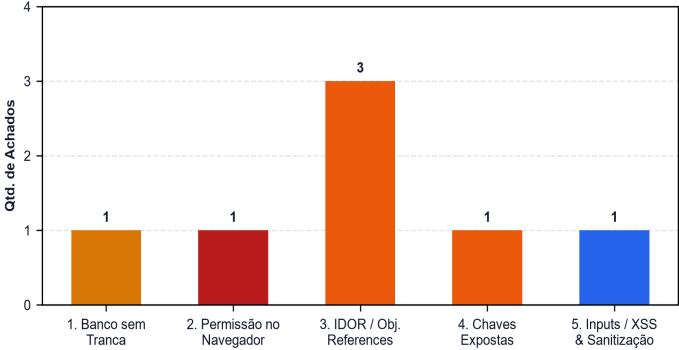
1. Resumo Executivo

A revisão de segurança identificou **9 achados** técnicos, categorizados em 1 Crítica, 3 Altas, 3 Médias e 2 Baixas/Informativas. O sistema apresenta uma postura louvável de **defesa em profundidade** no frontend e cabeçalhos HTTP, incluindo políticas modernas de segurança (CSP, HSTS, X-Frame-Options: DENY) e uso sistemático de `textContent` para evitar injeções XSS diretas no DOM. No entanto, o risco primordial reside no **desacoplamento de autorização entre o navegador e o webhook**: o fluxo pós-pagamento libera o atendimento VIP e marca o pedido como aprovado no cliente com base exclusivamente em parâmetros de URL (`?status=pago`), permitindo acesso não autorizado aos serviços sem comprovação financeira prévia.

Distribuição por Severidade



Achados por Categoria Auditada



2. Avaliação de Postura: Pontos Fortes e Riscos Centrais

Ponto Forte Verificado (Evidência Real)	Impacto Positivo na Segurança
Defesa Anti-XSS via DOM <code>textContent</code> : index.html:1167-1466	Todas as interpolações dinâmicas de texto (título da faixa, contadores, identificador de pedido, nome do cliente e homenageado) utilizam <code>element.textContent</code> ao invés de <code>innerHTML</code> , anulando a execução de scripts em injeções DOM-based.
Blindagem Completa de Headers HTTP: vercel.json:10-37	Implementação ativa de X-Frame-Options: DENY (proteção contra Clickjacking), Strict-Transport-Security (HSTS com preload), nosniff, Referrer-Policy e Permissions-Policy restritivo.
Validação Criptográfica de Formato & Rate Limit: api/check-status.js:7-68	A rota de checagem implementa Rate Limiting por IP (máx 60 req/min) e valida o <code>orderId</code> com expressão regular estrita (<code>/^HZ-\d+\$/</code>), barrando SQLi, path traversal e injeções de caracteres anômalos.
Idempotência e Não-Exposição em Webhook: api/asaas-webhook.js:53-64, 117-120	O webhook do Asaas possui cache de idempotência contra repetições fraudulentas e mascara erros internos, respondendo com mensagens genéricas sem vazamento de stack traces ou caminhos de servidor.

Riscos Centrais (Pontos Fracos):

- Bypass de Autorização Client-Side:** A liberação da UI de pagamento aprovado e do botão de atendimento WhatsApp baseia-se na presença de strings na URL, tornando o controle contornável por qualquer visitante.
- Sobrescrita Arbitrária de Pedidos (IDOR):** O backend aceita o identificador de pedido fornecido no corpo da requisição sem validar se a chave já existe, permitindo que terceiros sobrescrevam e invalidem pedidos de outros usuários.
- Token de Webhook em Código (Chave Exposta):** Fallback hardcoded em `api/asaas-webhook.js` exposto no histórico do repositório Git.

3. Tabela Detalhada de Achados por Categoria

Sev.	Arquivo : Linhas	Vulnerabilidade & Causa Raiz	Categoria
CRÍTICA	index.html L:1586-1620	Bypass de Autorização no Navegador: O estado de aprovação e o botão WhatsApp são liberados via parâmetro de URL (?status=pago ou #sucesso) sem confirmação do backend.	2. Permissão no Navegador
ALTA	api/create-order.js L:58-69	IDOR / Sobrescrita Arbitrária de Pedidos: A função aceita qualquer orderId enviado pelo cliente e sobrescreve o pedido existente no JSON sem verificar existência prévia.	3. IDOR
ALTA	api/asaas-webhook.js L:64	Segredo Padrão Hardcoded: Fallback público 'hizabellai_music_webhook_token_2036' permite forjar notificações caso a variável de ambiente falhe.	4. Chaves Expostas
ALTA	api/asaas-webhook.js L:93-102	Race Condition Lógica / Atribuição Cruzada: Pagamento sem referência externa é associado por aproximação de valor ao pedido mais recente, podendo aprovar pedido de usuário divergente.	3. IDOR / Lógica
MÉDIA	.htaccess L:1-14	Falta de Isolamento de Diretório no Apache: Não há bloqueio de leitura direta para /data/orders.json caso a aplicação seja hospedada em Apache/XAMPP.	1. Banco sem Tranca
MÉDIA	api/create-order.js L:43, 50-60	CORS Irrestrito e Ausência de Rate Limit: Access-Control-Allow-Origin: * e falta de controle de taxa em rota de escrita favorecem DoS por saturação de disco.	1. Isolamento & DoS
MÉDIA	api/check-status.js L:65-80	Enumeração de Pedidos por ID Preditivo: O uso de timestamp sequencial em HZ-timestamp permite que invasores mapeiem o volume e status de pedidos alheios.	3. IDOR (Leitura)
BAIXA	vercel.json L:35	CSP Permissivo ('unsafe-inline'): Diretiva de script permite execução inline, diminuindo o isolamento contra vetores de Cross-Site Scripting.	5. Inputs / XSS
INFO	index.html L:1504-1516	Sanitização Baseada em Expressões Regulares: Função artesanal com replace() simples em vez de biblioteca padrão de segurança (ex: DOMPurify).	5. Inputs / XSS

4. Plano de Ação e Recomendações Priorizadas

P1 (Imediato - Risco Financeiro): Remover a aprovação de status de pagamento baseada exclusivamente em parâmetros de URL no frontend (index.html). O desbloqueio do atendimento WhatsApp deve depender exclusivamente de validação assinada via HMAC ou confirmação assíncrona recebida no endpoint de checagem.

P2 (Crítico - Integridade & Credenciais): Eliminar o segredo default hardcoded em api/asaas-webhook.js, forçando a função a rejeitar inicialização se process.env.ASAAS_WEBHOOK_TOKEN não estiver configurado. No api/create-order.js, impedir que o cliente forneça o orderId, gerando UUID v4 exclusivo e imprevisível no backend.

P3 (Alto - Defesa em Profundidade): Remover a lógica de atribuição arbitrária por proximidade de valor no webhook. Se o pagamento não contiver externalReference correspondente, colocar o evento em quarentena ou notificar logs de auditoria.

P4 (Médio - Infraestrutura & DoS): Adicionar regra de bloqueio em .htaccess para impedir o acesso HTTP direto ao diretório /data/ e arquivos .json. Implementar Rate Limiting por IP em api/create-order.js e restringir o CORS da API à origem exata da plataforma.

5. Issues Acionáveis para o GitHub

Os blocos abaixo estão prontos para serem copiados diretamente para o GitHub Issues do repositório, fornecendo aos desenvolvedores todas as evidências, critérios de aceite e código de correção necessário.

--- ISSUE 1 ---
Título: [Segurança] Bypass de autorização de pagamento via query string no frontend
Labels sugeridas: security, severidade:crítica, auth
Descrição: O frontend da landing page libera a tela de pagamento aprovado e o botão do WhatsApp com status de pedido APROVADO ✓ se a URL contiver '?status=pago', '?status=approved' ou '#sucesso'. Qualquer usuário mal-intencionado pode navegar até esse link sem efetuar o pagamento no Asaas e forçar a liberação do atendimento.
Evidência: Arquivo: index.html (Linhas 1586-1620) Trecho: <pre>const isDirectPaid = params.get('status') === 'pago' params.get('status') === 'approved' ...; if (isDirectPaid) { pedido.status = 'PAID'; renderPaidSuccess(orderId, pedido); }</pre>
Impacto: Fraude financeira: clientes conseguem atendimento de produção musical sem confirmação de compensação bancária.
Sugestão de Correção: Exigir que a liberação do WhatsApp dependa exclusivamente da resposta da API 'api/check-status' (consultando o status gravado pelo webhook autêntico do Asaas), ou de um token de sessão assinado.
CrITÉrios de Aceite: - [] Remover validação direta de 'status=pago' para liberação do botão verde do WhatsApp - [] Manter o modal em estado PENDING até que 'api/check-status' retorne 'PAID' - [] Testar acesso manual a '?status=pago' e garantir que o botão WhatsApp não seja desbloqueado
--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] IDOR e sobrescrita arbitrária de pedidos em api/create-order.js**Labels sugeridas:** security, severidade:alta, idor

Descrição: O endpoint 'api/create-order.js' aceita o 'orderId' diretamente do corpo JSON da requisição e não verifica se a chave já existe no arquivo de pedidos. Um invasor pode enviar requisições com IDs de pedidos de outras pessoas e sobrescrever dados cadastrais ou reverter o status de PAID para PENDING.

Evidência:

Arquivo: api/create-order.js (Linhas 58-69)

Trecho:

```
const orders = readOrders();
orders[orderId] = {
  orderId,
  status: 'PENDING',
  cliente: cliente || {},
  ...
};
writeOrders(orders);
```

Impacto: Adulteração de dados, negação de serviço para clientes legítimos e quebra da integridade dos pedidos.

Sugestão de Correção: Gerar o 'orderId' obrigatoriamente no servidor utilizando 'crypto.randomUUID()' e rejeitar requisições de criação de pedidos cujo ID já esteja registrado.

Critérios de Aceite:

- [] Gerar ID único imprevisível no backend (UUID v4 / nanoid)
- [] Impedir sobrescrita de pedidos existentes retornando 409 Conflict se o ID já existir
- [] Validar schema e faixas de preço aceitáveis dos pacotes no backend

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] Segredo de autenticação padrão hardcoded em api/asaas-webhook.js**Labels sugeridas:** security, severidade:alta, credentials

Descrição: O webhook do Asaas possui um token de fallback estático codificado diretamente no arquivo ('hizabellai_music_webhook_token_2036'). Caso a variável de ambiente não seja injetada ou ocorra erro de carregamento no runtime serverless, qualquer agente externo pode enviar notificações forjadas de pagamento.

Evidência:

Arquivo: api/asaas-webhook.js (Linha 64)

Trecho:

```
const expectedToken = process.env.ASAAS_WEBHOOK_TOKEN || 'hizabellai_music_webhook_token_2036';
```

Impacto: Falsificação de eventos de pagamento confirmados, permitindo que atacantes aproveitem transações inexistentes.

Sugestão de Correção: Rejeitar a requisição com erro 500 caso a variável process.env.ASAAS_WEBHOOK_TOKEN não esteja definida. Rotacionar imediatamente o token atual e registrar um novo valor secreto exclusivamente nas variáveis da Vercel e Asaas.

Critérios de Aceite:

- [] Remover o valor default em texto plano do código-fonte
- [] Adicionar validação 'if (!process.env.ASAAS_WEBHOOK_TOKEN) throw new Error(...)'
- [] Rotacionar a credencial no painel do Asaas e na Vercel

--- FIM ISSUE 3 ---

--- ISSUE 4 ---
Título: [Segurança] Race condition e atribuição cruzada de pagamentos no webhook
Labels sugeridas: security, severidade:alta, business-logic
Descrição: Quando o Asaas envia um webhook sem externalReference válida, o handler executa um fallback que busca o pedido mais recente com status PENDING e valor monetário similar (+/- R\$ 0,50). Se dois pedidos forem gerados em momentos próximos com o mesmo plano, o pagamento de um cliente aprovará o pedido de outro.
Evidência: Arquivo: api/asaas-webhook.js (Linhas 93-102) Trecho: <pre>for (const k of keys) { if (orders[k].status === 'PENDING') { const val = parseFloat(orders[k].offerPrice 0); if (Math.abs(val - paymentVal) < 0.5) { targetOrderId = k; break; } } }</pre>
Impacto: Atribuição incorreta de pedidos, estornos e inconsistências financeiras graves.
Sugestão de Correção: Exigir estritamente que o link de pagamento ou cobrança do Asaas carregue a referência externa do pedido ou utilizar metadados persistentes, rejeitando aprovações heurísticas baseadas em aproximação de preço.
Critérios de Aceite: - [] Remover a lógica de fallback por aproximação de valor - [] Registrar pedidos não identificados em fila de auditoria manual - [] Assegurar que os links de checkout do Asaas passem o parâmetro de externalReference
--- FIM ISSUE 4 ---

--- ISSUE 5 ---
Título: [Segurança] Diretório /data/ exposto e sem restrição de acesso em ambiente Apache
Labels sugeridas: security, severidade:média, info-disclosure
Descrição: O arquivo .htaccess não contém regras para restringir a listagem ou o download de arquivos sensíveis na pasta 'data/'. Se o projeto for executado sob um servidor Apache padrão (como XAMPP), o arquivo 'data/orders.json' fica publicamente acessível para download direto pela web.
Evidência: Arquivo: .htaccess (Linhas 1-14) Ausência de diretivas de bloqueio a arquivos .json ou pasta /data/.
Impacto: Vazamento em massa de dados confidenciais de clientes (nome, telefone WhatsApp, mensagens pessoais e preferências musicais).
Sugestão de Correção: Inserir diretiva de bloqueio explícito no .htaccess: ' Require all denied '.
Critérios de Aceite: - [] Adicionar bloqueio de arquivos .json no .htaccess - [] Testar requisição direta a /data/orders.json e verificar retorno 403 Forbidden - [] Garantir que em produção na Vercel o diretório data não seja servido estaticamente
--- FIM ISSUE 5 ---